



RL Training for Dynamic Context Management in Mathematical Reasoning

Batu El¹, Mehmet Hamza Erol², Hannah Park-Kaufmann¹

¹Institute for Computational and Mathematical Engineering, Stanford University

²Department of Computer Science, Stanford University

Stanford

CS 224R - Deep RL

Problem Description

Problem:

- Language models have **limited context windows** and **degrade with longer context**.
- Teaching models to **backtrack improves reasoning** but remains **confined to in-context operations**.

Extension Motivation:

- We hypothesize that dynamic context management—**offloading irrelevant information from context** and **prioritizing the presence of relevant content in the context window**—can be facilitated by RL to enhance reasoning capabilities.

Proposed Extension (Solution):

- We propose **explicit backtracking for dynamic context management**, training models to **clean** their context when reasoning fails, enabling more effective retries.

Setup

Math reasoning (Countdown)

- Train the model to plan and execute the sequence of mathematical operations to reach a target number.
- Evaluating ability to decompose and carry out multi-step arithmetic reasoning.

Methodology

- Regular Supervised Fine-Tuning (SFT): Teach the model basic backtracking

$$\max_{\theta} \mathbb{E}_{x,y \in D} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t | x, y_{<t}) \right]$$

- Modified REINFORCE Leave-One-Out (RLOO): Let the model learn from experience & verifier-based rewards

References

- [1] Gandhi et al. 2024. Stream of Search (SoS): Learning to search in language. arXiv:2404.03683
[2] Rafailov et al. 2023. Direct preference optimization: Your LM is secretly a reward model. NeurIPS 36:53728–53741
[3] Jin et al. 2025. Search-R1: Training LLMs to reason and leverage search engines with RL. arXiv:2503.09516
[4] Packer et al. 2024. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560

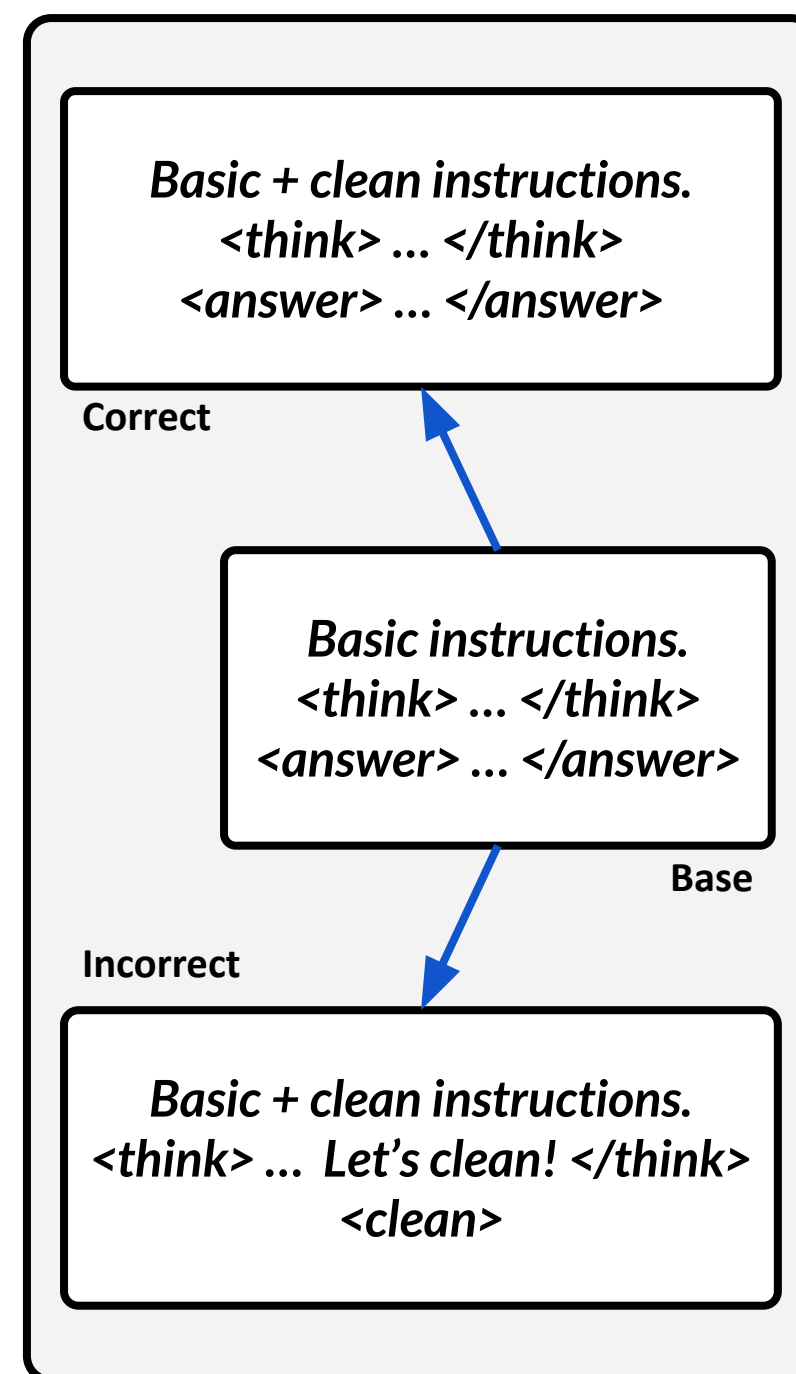
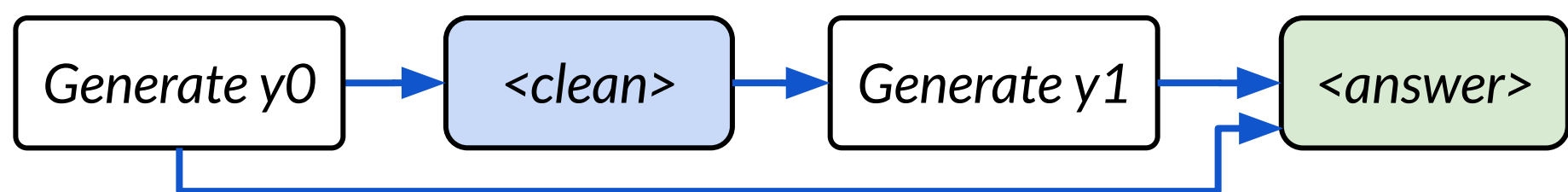
Our Approach

SFT Data Curation

- Reformat all data samples to be one think block followed by one answer block (fixing any incompletely formatted tags)
- Augment the data to teach the model to issue a `<clean>` tag
 - For incorrect completions, the model should learn to clean
 - For correct completions, the model should learn to answer

Context Management

- Instruct model for `<clean>` tags, then generate response
 - If model issues `<clean>` tag, store response, remove `<clean>` instructions, and reprompt.
 - If the model does not issue a `<clean>` tag, stop the generation



RLOO Formulation

$$A(x, y_i^0, y_i^1) = \left[R(x, y_i^0, y_i^1) - \frac{1}{k-1} \sum_{j \neq i} R(x, y_i^0, y_i^1) \right]$$
$$\frac{1}{K} \sum_{i=1}^K A(x, y_i^0, y_i^1) \left[\frac{\nabla \log \pi(y_i^0 | x)}{|y_i^0|} + \frac{\nabla \log \pi(y_i^1 | x, y_i^0)}{|y_i^1|} \right]$$

Reward is computed based on the final step of the generation.

The model learns to take the actions that lead to high rewards.

Experiment Results

	Default SFT	Default RLOO	Extension SFT	Extension RLOO
Accuracy	41.50	76.00	59.50	79.00
Score	47.00	77.10	62.65	80.25
Easy Score	54.33	90.18	70.98	89.76
Hard Score	13.61	17.50	24.72	36.94
Easy Count	164.00	164.00	164.00	164.00
Hard Count	36.00	36.00	36.00	36.00
Average Completion Length	916.56	676.22	1255.55	1117.95

- RLOO yields substantial gains over SFT across all metrics
- Extension models demonstrate superior performance** compared to default models, particularly on hard samples (includes at least multiplication or division: 18% of all samples)
- Extension models can generate longer sequences without performance degradation → effective context utilization
- SFT uses `<clean>` 31% of the time vs. 15.5% for SFT+RLOO, yet RLOO achieves better scores when cleaned, showing **more effective <clean> usage**.

SFT setup: 700 steps, batch size 64, learning rate: 2e-5

RLOO setup: 700 steps, batch size 16, learning rate: 1e-6

Discussion

Reasoning Samples

```
Let me analyze this step by step:
1. First, let's look for numbers close to
68 or factors/multiples that could help:
...
4. Let's try multiplication:
- 27 * 2 = 54 (close to 60)
- Not very helpful
5. New approach:
- 60 + 19 = 79 (too high)
- 60 - 27 = 33 (not helpful)
...
This works! Let's verify:
60 + 27 = 87
87 - 19 = 68
</think>
<answer> (60 + 27) - 19 </answer>
```

[60, 27, 19] → 68

Example CoT

```
...
After multiple attempts, I'm having trouble
reaching exactly 55 with these numbers
using only the allowed operations. Let me
try one final approach:

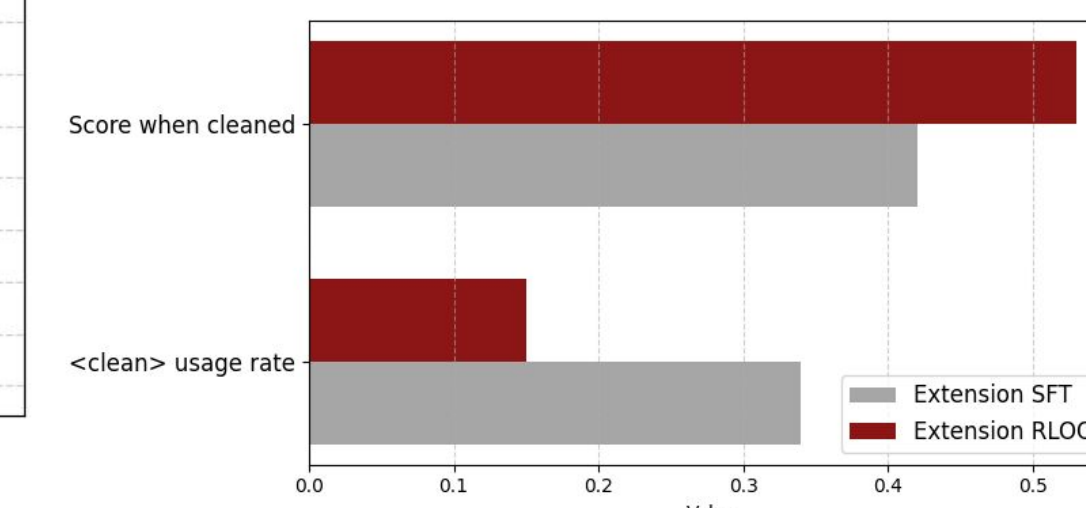
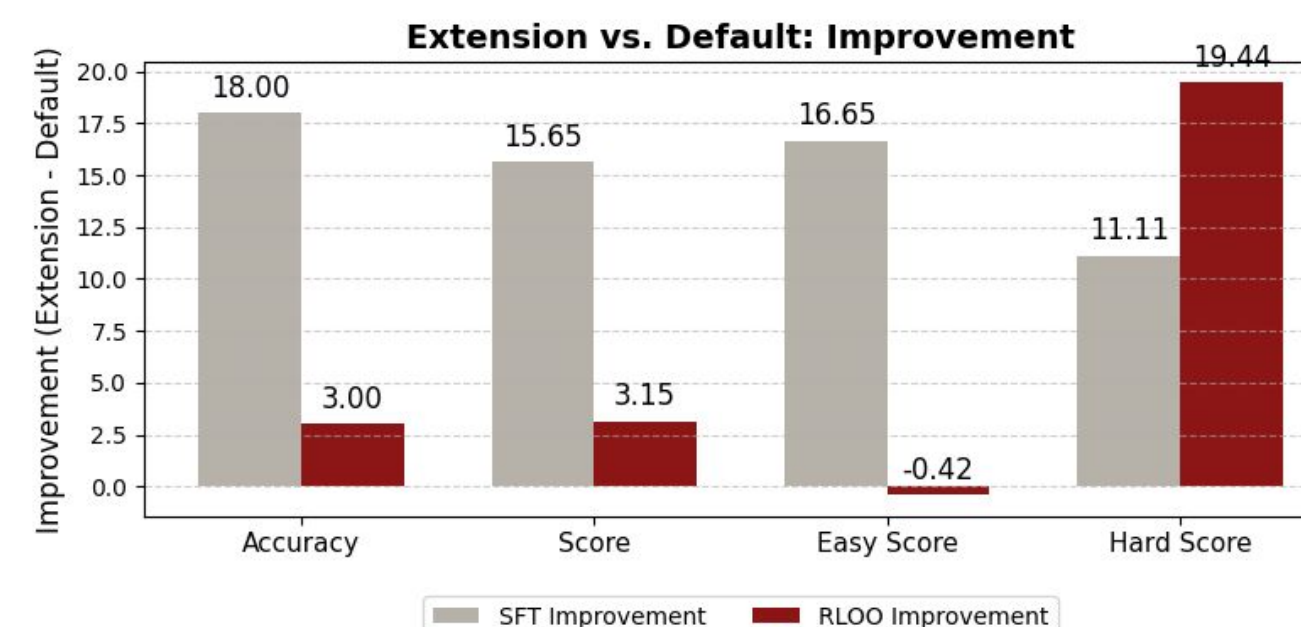
12. Let's try:
- 63 / 57 ≈ 1.08 (not helpful)
- 61 + 6 = 67
- 67 - 6 = 61 (not 55)

None of these attempts reach exactly 55 as
requested.
I realize my search is getting confusing.
Let me reset and start fresh.
</think>
<clean>
```

[61, 63, 57] → 55

Example Cleaning

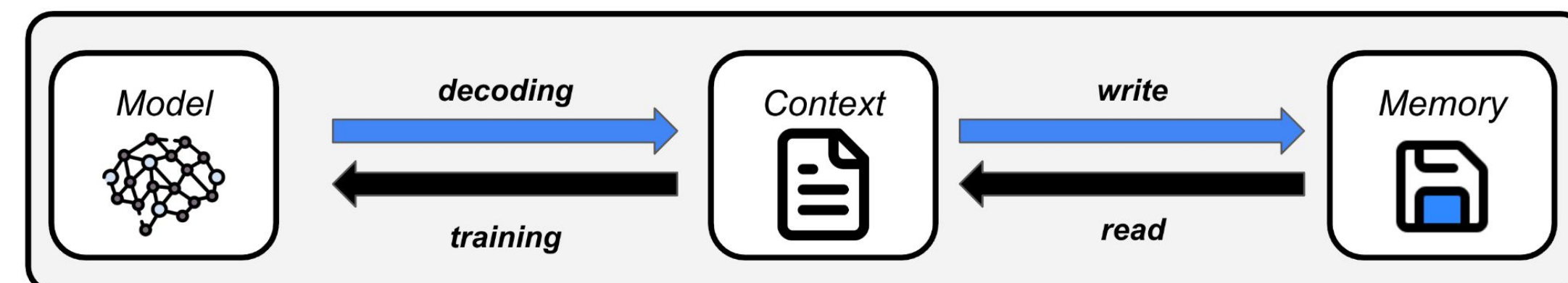
Analysis



Limitations and Future Work

Limitations:

- Cleaning the context removes all prior information, preventing the next generation from accessing the previous state. This is not ideal, as the model may repeat the same mistakes.
- For simplicity and computational efficiency, context cleaning is allowed only once.
- It remains unclear when the model should continue generating within the same context versus when it should reset its context.



Future Directions:

- Training a model with RLOO to use a memory with read and write operations for dynamic context management.